

Name-Nancy

Subject-computer vision Lab

CUID-CU240251005

✓ Experiment No. 5

Experiment Name :- Feature Extraction and Image Analysis using SIFT and HOG Descriptors

1. Import the required libraries and load a real-world image or image dataset for analysis.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage.feature import hog
from skimage import exposure
# Load the image
image = cv2.imread('/butterfly.webp')
# Convert BGR to RGB for displaying with Matplotlib
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
# Display image
plt.figure(figsize=(6, 8))
plt.imshow(image_rgb)
plt.title("Original Butterfly Image")
plt.axis("off")
plt.show()
```

Original Butterfly Image



2. Convert the image into grayscale, if required, and perform basic preprocessing to improve image quality.

```
# Convert image to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
plt.figure(figsize=(6, 8))
plt.imshow(gray, cmap='gray')
plt.title("Grayscale Image")
plt.axis("off")
plt.show()
```



3. Apply the SIFT (Scale-Invariant Feature Transform) algorithm to detect keypoints and compute feature descriptors.

```
# Create SIFT detector
sift = cv2.SIFT_create()
keypoints, descriptors = sift.detectAndCompute(gray, None)
print("Number of SIFT Keypoints:", len(keypoints))
print("SIFT Descriptor Shape:", descriptors.shape)
# Draw keypoints
sift_image = cv2.drawKeypoints(
    image_rgb,
    keypoints,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)
plt.figure(figsize=(8, 10))
plt.imshow(sift_image)
plt.title("SIFT Keypoints")
```

```
plt.axis("off")
plt.show()
```

Number of SIFT Keypoints: 2743
SIFT Descriptor Shape: (2743, 128)



4. Visualize the detected keypoints on the image and analyze their distribution across different image regions.

```
gray_float = gray.astype('float32') / 255.0
hog_features, hog_image = hog(
    gray_float,
    orientations=9,
    pixels_per_cell=(8, 8),
    cells_per_block=(2, 2),
    block_norm='L2-Hys',
    visualize=True
)

print("HOG Feature Vector Length:", len(hog_features))
```

HOG Feature Vector Length: 169128

5. Extract Histogram of Oriented Gradients (HOG) features from the same image using appropriate parameters such as cell size, block size, and orientation bins.

```
# Improve HOG visualization
hog_image_rescaled = exposure.rescale_intensity(
    hog_image,
    in_range=(0, 10)
```

)

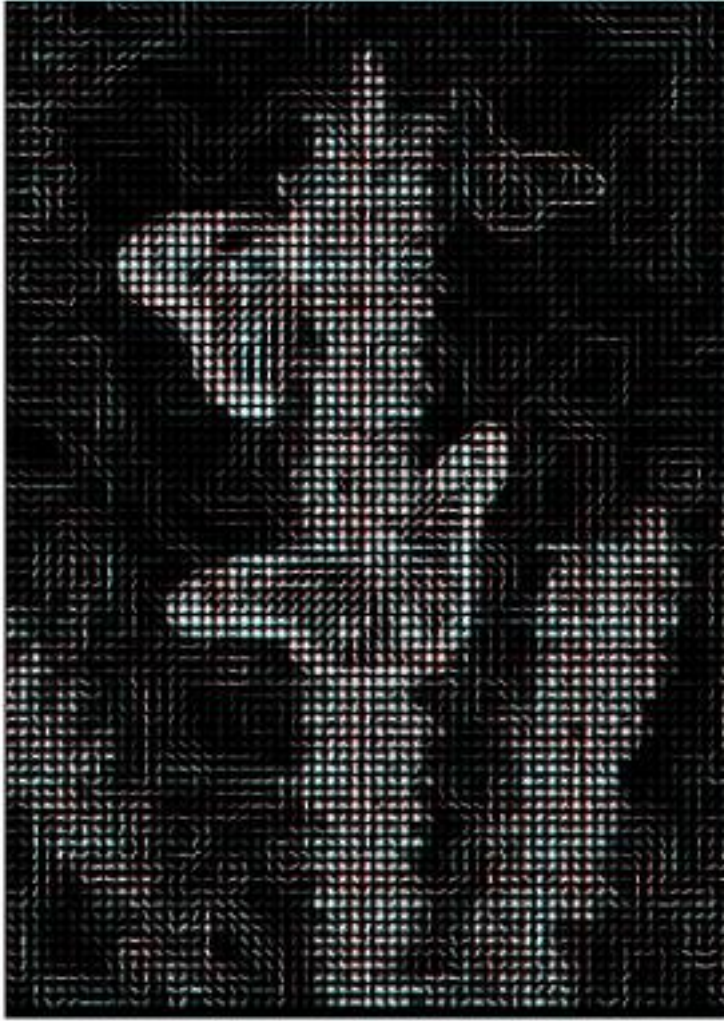
```
plt.figure(figsize=(8, 10))
plt.imshow(hog_image_rescaled, cmap='gray')
plt.title("HOG Feature Visualization")
plt.axis("off")
plt.show()
```



6. Visualize the HOG descriptor to understand the gradient orientation patterns captured from the image.

```
# Display HOG descriptor
plt.figure(figsize=(7, 9))
plt.imshow(hog_image_rescaled, cmap='gray')
plt.title("HOG Descriptor Visualization")
plt.axis("off")
plt.show()
```

HOG Descriptor Visualization



7. Compare the characteristics of SIFT keypoints and HOG descriptors with respect to scale invariance, rotation invariance, and computational efficiency.

```
import cv2
import time
from skimage.feature import hog
image = cv2.imread('/butterfly.webp')
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
start_time = time.time()
sift = cv2.SIFT_create()
keypoints, descriptors = sift.detectAndCompute(gray, None)
sift_time = time.time() - start_time
start_time = time.time()
hog_features = hog(
    gray,
    orientations=9,
    pixels_per_cell=(8, 8),
    cells_per_block=(2, 2),
    block_norm='L2-Hys'
)

hog_time = time.time() - start_time
print("SIFT vs HOG")
print("\nSIFT:")
print("Number of Keypoints:", len(keypoints))
print("Descriptor Shape:", descriptors.shape)
print("Computational Time:", sift_time, "seconds")

print("\nHOG:")
print("Feature Vector Length:", len(hog_features))
print("Computational Time:", hog_time, "seconds")
```

```

print("\n CHARACTERISTICS ")

print("SIFT:")
print("- Scale Invariant: Yes")
print("- Rotation Invariant: Yes")
print("- Computational Efficiency: Relatively slower")

print("\nHOG:")
print("- Scale Invariant: No")
print("- Rotation Invariant: Limited")
print("- Computational Efficiency: Relatively faster")

```

SIFT vs HOG

SIFT:
Number of Keypoints: 2743
Descriptor Shape: (2743, 128)
Computational Time: 0.19961166381835938 seconds

HOG:
Feature Vector Length: 169128
Computational Time: 0.1664433479309082 seconds

CHARACTERISTICS

SIFT:

- Scale Invariant: Yes
- Rotation Invariant: Yes
- Computational Efficiency: Relatively slower

HOG:

- Scale Invariant: No
- Rotation Invariant: Limited
- Computational Efficiency: Relatively faster

8. Apply SIFT or HOG features to perform basic image matching or object recognition using two similar images.

```

bf = cv2.BFMatcher()
matches = bf.knnMatch(descriptors, descriptors2, k=2)
good_matches = []
for m, n in matches:
    if m.distance < 0.7 * n.distance:
        good_matches.append(m)

print("Total Matches:", len(matches))
print("Good Matches:", len(good_matches))
good_matches = sorted(good_matches, key=lambda x: x.distance)
matched_image = cv2.drawMatches(
    image_rgb,
    keypoints,
    cv2.cvtColor(image2, cv2.COLOR_BGR2RGB),
    keypoints2,
    good_matches[:50],
    None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)
plt.figure(figsize=(16, 9))
plt.imshow(matched_image)
plt.title("SIFT Feature Matching")
plt.axis("off")
plt.show()

```

Total Matches: 2743
Good Matches: 1226

SIFT Feature Matching



9. Evaluate the strengths and limitations of both feature extraction techniques for different computer vision tasks.

```
import cv2
import matplotlib.pyplot as plt
image = cv2.imread('/butterfly.webp')
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

sift = cv2.SIFT_create()
keypoints, descriptors = sift.detectAndCompute(gray, None)

print("Number of keypoints:", len(keypoints))
print("Descriptor shape:", descriptors.shape)

result = cv2.drawKeypoints(
    image_rgb,
    keypoints,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

plt.figure(figsize=(8, 10))
plt.imshow(result)
plt.title("SIFT Keypoints")
plt.axis("off")
plt.show()
```

Number of keypoints: 2743
Descriptor shape: (2743, 128)

SIFT Keypoints



10. Document the observations and discuss suitable applications where SIFT and HOG descriptors can be effectively utilized.

```
features, hog_image = hog(  
    gray,  
    orientations=9,  
    pixels_per_cell=(8, 8),  
    cells_per_block=(2, 2),  
    block_norm='L2-Hys',  
    visualize=True  
)  
print("HOG feature vector length:", len(features))  
hog_image = exposure.rescale_intensity(  
    hog_image,  
    in_range=(0, 10)  
)  
  
plt.figure(figsize=(8, 10))  
plt.imshow(hog_image, cmap='gray')  
plt.title("HOG Feature Visualization")  
plt.axis("off")  
plt.show()
```

HOG feature vector length: 169128

HOG Feature Visualization

